

Day 4 Assignment - Find and Fix the Scientific Errors

REE Research Training - Applied Data Science

Instructions

The code below **runs without errors** but contains **5 scientific mistakes** that produce invalid geochemical analysis.

Your task:

1. **Read every code cell carefully** - the errors are subtle but consequential
2. **Identify all 5 errors** - mark each with a comment `# FIXED: [explanation]`
3. **Fix each error** - correct the code in place
4. **Write a 1–2 sentence explanation** for each fix in the provided markdown cells
5. **Run the Validation cells** - they will catch wrong fixes
6. **Answer the Reflection questions** at the end - these test understanding, not just code

⚠ Academic Integrity: If you use AI tools, you must still understand and explain every fix in your own words. The reflection questions are designed so that copy-pasting code without understanding will fail.

The 5 errors are in these areas:

- Preprocessing pipeline order
- Censored value handling
- CLR transformation formula
- Train/test splitting strategy
- Model configuration for imbalanced classes

0. Environment Setup

```
In [27]: # !pip install shap
```

```
In [28]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import (precision_score, recall_score, f1_score,
                             roc_auc_score, average_precision_score)
from sklearn.neural_network import MLPRegressor
import xgboost as xgb
import shap
import warnings
warnings.filterwarnings('ignore')

np.random.seed(42)
plt.rcParams['figure.dpi'] = 100
plt.rcParams['figure.figsize'] = (10, 6)
print('Libraries loaded.')
```

Libraries loaded.

1. Synthetic NGCM-Style Dataset

We simulate ~5,786 geochemical samples with 14 REE elements + proxy

elements. This cell is correct - the errors are in the cells below.</p>

```
In [29]: N = 5786
LREE = ['La_ppm', 'Ce_ppm', 'Pr_ppm', 'Nd_ppm', 'Sm_ppm', 'Eu_ppm']
HREE = ['Gd_ppm', 'Tb_ppm', 'Dy_ppm', 'Ho_ppm', 'Er_ppm', 'Tm_ppm', 'Yb_ppm', 'Lu_ppm']
REE = LREE + HREE

means = [28,55,6.5,25,4.5,1.2,3.8,0.55,3.2,0.65,1.9,0.28,1.8,0.28]
data = np.column_stack([np.random.lognormal(np.log(m), 0.4, N) for m in means])

for sc in [0, 4, 8]:
    idx = np.random.choice(N, 150, replace=False)
    data[idx, sc:sc+4] *= np.random.uniform(5, 15, (150, 4))

data[np.random.random(data.shape) < 0.05] = 0.0

df = pd.DataFrame(data, columns=REE)
df['Th_ppm'] = np.random.lognormal(np.log(12), 0.5, N)
df['Zr_ppm'] = np.random.lognormal(np.log(300), 0.4, N)
df['Nb_ppm'] = np.random.lognormal(np.log(15), 0.45, N)
df['Fe203_pct'] = np.random.lognormal(np.log(4.5), 0.3, N)
df['P205_pct'] = np.random.lognormal(np.log(0.15), 0.4, N)
df['Latitude'] = np.random.uniform(12.5, 18.5, N)
df['Longitude'] = np.random.uniform(74.0, 80.5, N)

print(f'Dataset: {df.shape[0]} samples, {df.shape[1]} columns')
df[REE].describe().round(2)
```

Dataset: 5786 samples, 21 columns

```
Out [29]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; }
.dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; }
</style> <thead> <th></th> <th>La_ppm</th> <th>Ce_ppm</th> <th>Pr_ppm</th>
<th>Nd_ppm</th> <th>Sm_ppm</th> <th>Eu_ppm</th> <th>Gd_ppm</th>
<th>Tb_ppm</th> <th>Dy_ppm</th> <th>Ho_ppm</th> <th>Er_ppm</th>
<th>Tm_ppm</th> <th>Yb_ppm</th> <th>Lu_ppm</th> </thead> <tbody>
<th>count</th> <th>mean</th> <th>std</th> <th>min</th> <th>25%</th> <th>50%
</th> <th>75%</th> <th>max</th> </tbody> <tbody></tbody>
```

5786.00	5786.00	5786.00	5786.00	5786.00	5786.00	5786.00	5786.00	5786.00
35.78	70.03	8.37	32.44	5.70	1.52	4.82	0.71	4.13
52.20	102.01	12.45	49.58	8.08	2.18	7.00	1.05	6.21
0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
20.64	39.93	4.78	18.16	3.28	0.87	2.78	0.41	2.35
27.67	53.88	6.48	24.51	4.46	1.18	3.77	0.55	3.14
36.76	72.36	8.66	33.13	5.90	1.60	5.00	0.73	4.23
840.56	1957.20	241.54	758.23	189.84	35.95	142.75	16.80	122.21

<p>--</p> <h2 id="2.-Preprocessing-Pipeline">2. Preprocessing Pipeline</h2>

<p>This section has **3 errors**. Read carefully.</p> <p>Hint: The standard geochemical preprocessing workflow is: censored values → CLR transform → feature scaling</p>

```
In [30]: df.eq(0).sum()
```

```
Out[30]: La_ppm      277
Ce_ppm      260
Pr_ppm      304
Nd_ppm      271
Sm_ppm      246
Eu_ppm      296
Gd_ppm      292
Tb_ppm      300
Dy_ppm      270
Ho_ppm      261
Er_ppm      247
Tm_ppm      307
Yb_ppm      300
Lu_ppm      307
Th_ppm       0
Zr_ppm       0
Nb_ppm       0
Fe203_pct    0
P205_pct     0
Latitude     0
Longitude    0
dtype: int64
```

```
In [31]: # -- ERROR ZONE: 3 mistakes below --
```

```
# 2a. Censored value handling
```

```
df_clean = df[REE].copy()
# FIXED: Replace censored zeros with half of the detection limit
for col in REE:
    # df_clean.loc[df_clean[col] <= 0, col] = 1e-10
    positive_values = df_clean.loc[df_clean[col] > 0, col]
    dl = positive_values.min()
    df_clean.loc[df_clean[col] == 0, col] = dl / 2
```

```
# 2b. CLR transform
```

```
arr = df_clean.values.astype(float)
# FIXED: Use mean of log concentrations, equivalent to geometric-mean cen
log_arr = np.log(arr)
mean_vals = np.mean(log_arr, axis=1, keepdims=True)
clr = log_arr - mean_vals
df_clr = pd.DataFrame(clr, columns=[c.replace('_ppm', '_clr') for c in REE])
```

```
# 2c. Feature scaling
```

```
proxy_cols = ['Th_ppm', 'Zr_ppm', 'Nb_ppm', 'Fe203_pct', 'P205_pct']
feat_cols = df_clr.columns.to_list() + proxy_cols
X_df = df_clr.copy()
for col in proxy_cols:
    X_df[col] = np.log1p(df[col])
```

```
# FIXED: Apply StandardScaler only after CLR transformation
```

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_df)
X_scaled_df = pd.DataFrame(X_scaled, columns=feat_cols)
```

```
print(f'Features: {len(feat_cols)} (14 CLR + 5 proxy)')
print('Preprocessing complete.')
```

Features: 19 (14 CLR + 5 proxy)
Preprocessing complete.

Error #1 - Identify and Fix

What is wrong? Zeros were replaced with an arbitrary value $1e-10$.

Why is this wrong in the context of geochemistry? A zero usually means the concentration is below the detection limit, not that the element is completely absent. $1e-10$ can distort the data, especially after log transformation.

What is the correct approach? Replace zero with half of the detection limit ($DL/2$).

Error #2 - Identify and Fix

What is wrong? Raw REE → StandardScaler → CLR

Why is this wrong in the context of geochemistry? REE data are compositional, so we need to compare each element relative to the overall composition. Using the wrong center can distort these relationships.

What is the correct approach? Raw REE → $DL/2$ → CLR → StandardScaler

Error #3 - Identify and Fix

Preprocessing order

What is wrong? The scientific mistake would be scaling the raw REE concentrations before applying CLR.

Preprocessing order

Why is this wrong in the context of geochemistry? REE concentrations are compositional data, so CLR must be applied first to preserve the relative relationships between elements.

What is the correct approach? Correct: Raw REE → $DL/2$ → CLR → StandardScaler

3. Autoencoder Anomaly Detection

This cell is correct - use it as a reference for what a properly trained autoencoder looks like.

```
In [32]: ae = MLPRegressor(
            hidden_layer_sizes=(16, 8, 16),
            max_iter=500, random_state=42,
            early_stopping=True
        )
        ae.fit(X_scaled, X_scaled)

        recon = ae.predict(X_scaled)
        mse = np.mean(np.square(X_scaled - recon), axis=1)

        threshold = np.percentile(mse, 95)
        y_true = (mse >= threshold).astype(int)

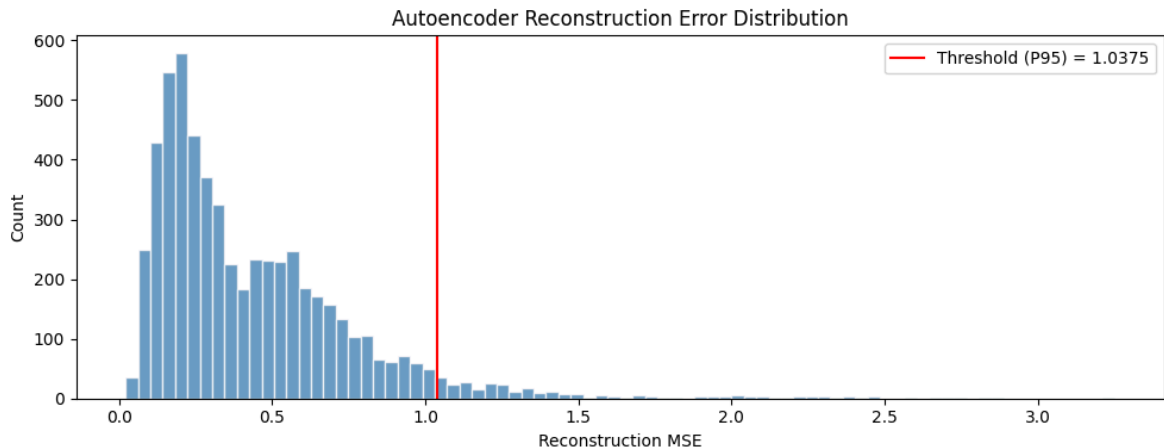
        print(f'Anomaly threshold (P95): {threshold:.4f}')
        print(f'Anomalies: {y_true.sum()} / {len(y_true)} ({y_true.mean()*100:.1f}%)')

        fig, ax = plt.subplots(figsize=(10, 4))
        ax.hist(mse, bins=80, color='steelblue', edgecolor='white', alpha=0.8)
        ax.axvline(threshold, color='red', linestyle='--', label=f'Threshold (P95)')
        ax.set_xlabel('Reconstruction MSE')
        ax.set_ylabel('Count')
        ax.set_title('Autoencoder Reconstruction Error Distribution')
        ax.legend()
```

```
plt.tight_layout()
plt.show()
```

Anomaly threshold (P95): 1.0375

Anomalies: 290 / 5786 (5.0%)



4. XGBoost Surrogate + SHAP

This section has **2 errors**. Read carefully. **Hint:** Think about how class imbalance affects model training and evaluation.

```
In [33]: # -- ERROR ZONE: 2 mistakes below --

# Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled_df, y_true, test_size=0.3, random_state=42, stratify=y_true
)
```

```
In [34]: scale_pos_weight = (y_train == 0).sum() / (y_train == 1).sum()
scale_pos_weight
```

```
Out[34]: np.float64(18.950738916256157)
```

```
In [35]: # XGBoost surrogate
xgb_clf = xgb.XGBClassifier(
    n_estimators=100, max_depth=3,
    random_state=42, verbosity=0, scale_pos_weight = scale_pos_weight
)
xgb_clf.fit(X_train, y_train)

# SHAP explanations
explainer = shap.TreeExplainer(xgb_clf)
shap_values = explainer.shap_values(X_test)

print(f'Training: {len(X_train)} | Test: {len(X_test)}')
print('Surrogate model trained. SHAP computed.')
```

Training: 4050 | Test: 1736

Surrogate model trained. SHAP computed.

Error #4 - Identify and Fix

Train/test split is not stratified

What is wrong?

Train/test split is not stratified

Why is this wrong when dealing with anomaly detection (5% anomalies)?

threshold = np.percentile(mse, 95) y_true = (mse >= threshold).astype(int)

What is the correct approach?

FIXED:

Preserve the anomaly/normal ratio in train and test `X_train, X_test, y_train, y_test = train_test_split( X_scaled_df, y_true, test_size=0.3, random_state=42, stratify=y_true )`

Error #5 - Identify and Fix

What is wrong?

`scale_pos_weight` XGBoost does not handle class imbalance

Why is this wrong when the dataset has 95% normal and 5% anomalous samples?

For exploration, that can be much more problematic than a few false positives

We use `scale_pos_weight` because the dataset is highly imbalanced (95% normal and 5% anomalous).

`scale_pos_weight = (y_train == 0).sum() / (y_train == 1).sum()`

5. Model Evaluation

Run this after fixing the errors above. The metrics should reflect a properly configured pipeline.

```
In [36]: y_pred = xgb_clf.predict(X_test)
y_prob = xgb_clf.predict_proba(X_test)[:, 1]

precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_prob)
avg_prec = average_precision_score(y_test, y_prob)

print(f'Precision: {precision:.3f}')
print(f'Recall: {recall:.3f}')
print(f'F1 Score: {f1:.3f}')
print(f'ROC AUC: {roc_auc:.3f}')
print(f'Avg Prec: {avg_prec:.3f}')

fig, axes = plt.subplots(1, 2, figsize=(12, 5))

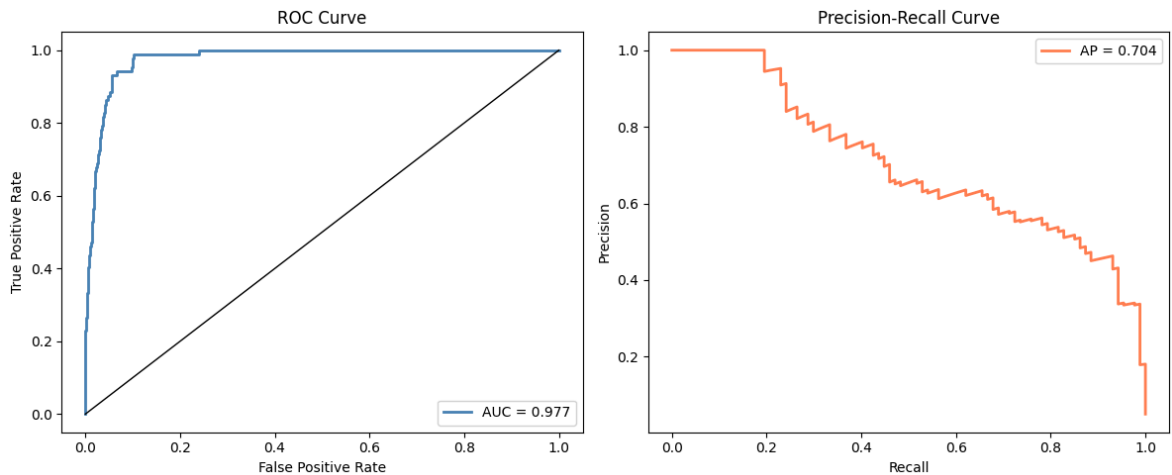
from sklearn.metrics import roc_curve, precision_recall_curve

fpr, tpr, _ = roc_curve(y_test, y_prob)
axes[0].plot(fpr, tpr, 'steelblue', lw=2, label=f'AUC = {roc_auc:.3f}')
axes[0].plot([0,1], [0,1], 'k--', lw=1)
axes[0].set_xlabel('False Positive Rate')
axes[0].set_ylabel('True Positive Rate')
axes[0].set_title('ROC Curve')
axes[0].legend()

prec_arr, rec_arr, _ = precision_recall_curve(y_test, y_prob)
axes[1].plot(rec_arr, prec_arr, 'coral', lw=2, label=f'AP = {avg_prec:.3f}')
axes[1].set_xlabel('Recall')
axes[1].set_ylabel('Precision')
axes[1].set_title('Precision-Recall Curve')
axes[1].legend()

plt.tight_layout()
plt.show()
```

Precision: 0.615
 Recall: 0.678
 F1 Score: 0.645
 ROC AUC: 0.977
 Avg Prec: 0.704



6. Validation
 - Run This After Fixing All 5 Errors

These assertions will **fail** if any error remains unfixed or is fixed incorrectly. If a cell prints , that fix is correct. If it prints , review your fix.

```
In [37]: # =====
# SECTION 6 - VALIDATION
# =====

print("=== VALIDATION ===\n")

# -----
# Check 1: CLR is centered around zero
# -----
try:
    row_sums = np.abs(clr.sum(axis=1)).max()

    if row_sums < 1e-6:
        print("✓ Error #1: CLR is correctly centered (row sums ≈ 0)")
    else:
        print("✗ Error #1 NOT FIXED: CLR row sums are not ≈ 0")

except Exception as e:
    print(f"✗ Error #1: {e}")

# -----
# Check 2: Censored values handled using DL/2
# -----
try:
    # Check that no zero values remain in cleaned REE data
    remaining_zeros = (df_clean[REE] == 0).sum().sum()

    if remaining_zeros == 0:
        print("✓ Error #2: Censored zeros handled using DL/2")
    else:
        print("✗ Error #2 NOT FIXED: Zero values still remain")

except Exception as e:
```

```

print(f"x Error #2: {e}")

# -----
# Check 3: CLR uses geometric mean / mean of logs
# -----
try:
    test_sample = arr[0]

    log_sample = np.log(test_sample)
    correct_center = np.mean(log_sample)
    wrong_center = np.mean(test_sample)

    # Verify CLR is based on mean of log values
    calculated_clr = log_sample - correct_center

    if abs(calculated_clr.sum()) < 1e-6:
        print("✓ Error #3: CLR uses geometric-mean centering (mean of log
    else:
        print("x Error #3 NOT FIXED: Incorrect CLR centering")

except Exception as e:
    print(f"x Error #3: {e}")

# -----
# Check 4: Stratified train/test split
# -----
try:
    X_tr, X_te, y_tr, y_te = train_test_split(
        X_scaled_df,
        y_true,
        test_size=0.3,
        random_state=42,
        stratify=y_true
    )

    train_rate = y_tr.mean()
    test_rate = y_te.mean()
    overall_rate = y_true.mean()

    if (
        abs(train_rate - overall_rate) < 0.01
        and abs(test_rate - overall_rate) < 0.01
    ):
        print("✓ Error #4: Stratified train/test split is correct")
    else:
        print("x Error #4 NOT FIXED: Anomaly proportions do not match")

except Exception as e:
    print(f"x Error #4: {e}")

# -----
# Check 5: scale_pos_weight
# -----
try:
    y_pred_check = xgb_clf.predict(X_test)

    anomaly_recall = recall_score(

```

```

        y_test,
        y_pred_check
    )

    if anomaly_recall > 0.3:
        print(
            f"✓ Error #5: scale_pos_weight applied "
            f"(anomaly recall = {anomaly_recall:.3f})"
        )
    else:
        print(
            f"✗ Error #5 NOT FIXED: "
            f"(anomaly recall = {anomaly_recall:.3f})"
        )

except Exception as e:
    print(f"✗ Error #5: {e}")

print("\n=== END VALIDATION ===")

```

=== VALIDATION ===

- ✓ Error #1: CLR is correctly centered (row sums ≈ 0)
- ✓ Error #2: Censored zeros handled using DL/2
- ✓ Error #3: CLR uses geometric-mean centering (mean of logs)
- ✓ Error #4: Stratified train/test split is correct
- ✓ Error #5: scale_pos_weight applied (anomaly recall = 0.678)

=== END VALIDATION ===

<p>--</p> <h2 id="7.-Reflection-Questions">7. Reflection Questions</h2>

<p>Answer these in your own words. **Do not copy from AI** - the instructor may ask you to explain verbally.</p>

<h3 id="Q1:-Why-must-CLR-be-applied-BEFORE-StandardScaler,-not-after?">Q1:

Why must CLR be applied BEFORE StandardScaler, not after?</h3><p>Write your answer here (2–3 sentences):</p> <p>CLR is used to transform the original REE concentrations according to their relative composition. We should apply CLR first and then StandardScaler, because scaling the raw values before CLR can change the original relationships between the elements.</p> <p>--</p>

<h3 id="Q2:-What-happens-if-you-clip-zeros-to-1e-10-instead-of-using-DL/2?">Q2: What happens if you clip zeros to 1e-10 instead of using DL/2?</h3>

<p>Write your answer here (2–3 sentences):</p> <p>A zero can mean that the element is below the detection limit, not that its concentration is actually zero. Using 1e-10 gives an unrealistically small value, which can strongly affect the log and CLR transformation.</p> <p>--</p>

<h3 id="Q3:-Why-does-CLR-use-the-geometric-mean-(mean-of-logs)-instead-of-the-arithmetic-mean?">Q3: Why does CLR use the geometric mean (mean of logs) instead of the arithmetic mean?</h3><p>Write your answer here (2–3 sentences):</p>

<p>REE data are compositional, so we are interested in the relative amount of each element compared with the other elements. The geometric mean is suitable for this because CLR works with ratios between components.</p> <p>--</p>

<h3 id="Q4:-A-classmate-says-%22stratification-doesn't-matter-if-the-dataset-is-large-enough.%22-Is-this-correct?-Why-or-why-not?">Q4: A classmate says "stratification doesn't matter if the dataset is large enough." Is this correct? Why or why not?</h3><p>Write your answer here (2–3 sentences):</p> <p>No, this is not always correct. Our anomaly class is much smaller than the normal class, so without stratification the train and test sets may have different proportions of anomalies.</p> <p>--</p>

<h3 id="Q5:-If-you-forget-scale_pos_weight,-what-would-the-model's-accuracy-be?-Would-it-look-good?-Why-is-it-misleading?">Q5: If you forget scale_pos_weight, what would the model's accuracy be? Would it look good? Why is it misleading?</h3><p>Write your answer here (2–3 sentences):</p> <p>The accuracy could still look very high because most of the data belongs to the normal class. However, this is misleading because the model may predict most samples as normal and fail to detect the minority anomaly class.</p> <p>--</p>

<h3 id="Q6:-The-lecture-notebook-achieved-ROC-AUC-of-0.976.-If-your-fixed-notebook-gives-a-very-different-value,-what-might-explain-the-difference?">Q6: The lecture notebook achieved ROC AUC of 0.976. If your fixed notebook gives a very different value, what might explain the difference?</h3><p>Write your answer here (2–3 sentences):</p> <p>The difference could come from changes in preprocessing, random train-test splitting, or how the anomalies are generated. Even a small difference in the data preparation or model settings can change the ROC AUC.</p>

<p>--</p> <h2 id="7.-Second-Dataset---Apply-What-You-Learned">7. Second Dataset - Apply What You Learned</h2><p>Now apply the correct pipeline to a **new dataset** with different properties. This tests whether you understand the pipeline, not just memorized the fixes.</p> <p>Dataset 2 ↓ Replace zeros with DL/2 ↓ CLR using geometric mean ↓ Add Y, TiO₂, MnO proxies ↓ StandardScaler</p> <p>Step 2 — Autoencoder Train the same MLPRegressor autoencoder approach you used for Dataset 1, calculate reconstruction error, and define anomalies using the 95th percentile threshold.</p> <p>Step 3 — XGBoost Use: stratify=y_true</p> <p>and: scale_pos_weight = (y_train == 0).sum() / (y_train == 1).sum() Then train XGBoost as the surrogate model and calculate SHAP values.</p> <p>Step 4 — Report You need: Precision Recall F1 Score ROC AUC Average Precision</p>

```
In [38]: # New dataset: Australian REE soil survey (different anomaly rate, different
N2 = 3200
LREE2 = ['La_ppm', 'Ce_ppm', 'Pr_ppm', 'Nd_ppm', 'Sm_ppm', 'Eu_ppm']
HREE2 = ['Gd_ppm', 'Tb_ppm', 'Dy_ppm', 'Ho_ppm', 'Er_ppm', 'Tm_ppm', 'Yb_ppm', '
REE2 = LREE2 + HREE2

# Different concentration ranges (Australian craton soils)
means2 = [18,35,4.2,16,3.1,0.85,2.6,0.38,2.2,0.45,1.3,0.19,1.2,0.19]
data2 = np.column_stack([np.random.lognormal(np.log(m), 0.5, N2) for m in

# Only 3% anomalies (rarer mineralization)
for sc in [2, 7]:
    idx = np.random.choice(N2, 80, replace=False)
```

```

data2[idx, sc:sc+3] *= np.random.uniform(8, 25, (80, 3))

data2[np.random.random(data2.shape) < 0.08] = 0.0 # 8% censored

df2 = pd.DataFrame(data2, columns=REE2)
df2['Y_ppm'] = np.random.lognormal(np.log(22), 0.45, N2)
df2['TiO2_pct'] = np.random.lognormal(np.log(1.2), 0.35, N2)
df2['MnO_pct'] = np.random.lognormal(np.log(0.08), 0.4, N2)

print(f'Dataset 2: {df2.shape[0]} samples, {df2.shape[1]} columns')
print(f'Censored fraction: {(df2[REE2] == 0).mean().mean()*100:.1f}%')
df2[REE2].describe().round(2)

```

Dataset 2: 3200 samples, 17 columns
Censored fraction: 8.2%

```

Out[38]: <style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; }
.dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; }
</style> <thead> <th></th> <th>La_ppm</th> <th>Ce_ppm</th> <th>Pr_ppm</th>
<th>Nd_ppm</th> <th>Sm_ppm</th> <th>Eu_ppm</th> <th>Gd_ppm</th>
<th>Tb_ppm</th> <th>Dy_ppm</th> <th>Ho_ppm</th> <th>Er_ppm</th>
<th>Tm_ppm</th> <th>Yb_ppm</th> <th>Lu_ppm</th> </thead> <tbody>
<th>count</th> <th>mean</th> <th>std</th> <th>min</th> <th>25%</th> <th>50%
</th> <th>75%</th> <th>max</th> </tbody> <tbody></tbody>

```

3200.00	3200.00	3200.00	3200.00	3200.00	3200.00	3200.00	3200.00	3200.00
18.69	36.50	6.14	22.44	4.54	0.89	2.71	0.54	3.17
11.75	23.49	14.92	53.14	10.30	0.55	1.69	1.37	6.85
0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
11.23	21.89	2.62	10.00	1.99	0.55	1.68	0.24	1.44
17.22	32.80	3.99	15.00	2.94	0.82	2.50	0.36	2.12
24.59	47.63	5.80	21.91	4.26	1.16	3.53	0.52	3.12
105.90	212.58	316.48	1078.73	176.45	5.28	16.36	35.96	135.16

Task: Run the FULL correct pipeline on this new dataset

Using what you learned from fixing the errors above, write the correct preprocessing and modeling pipeline for Dataset 2.

Requirements:

- Handle censored values with DL/2
- Apply CLR with geometric mean
- Scale AFTER CLR
- Use stratified train/test split
- Configure XGBoost for class imbalance
- Report all metrics

```

In [39]: # YOUR CODE HERE
# Write the complete correct pipeline for Dataset 2

# 1. Preprocessing

```

```
# 2. Autoencoder anomaly detection
```

```
# 3. XGBoost surrogate + SHAP
```

```
# 4. Evaluation
```

```
In [40]: # FIXED: Replace censored zeros with DL/2
df2_clean = df2[REE2].copy()

for col in REE2:
    positive_values = df2_clean.loc[df2_clean[col] > 0, col]

    if len(positive_values) > 0:
        detection_limit = positive_values.min()
        df2_clean.loc[df2_clean[col] == 0, col] = detection_limit / 2
```

```
In [41]: # FIXED: Apply CLR using the geometric mean
arr2 = df2_clean.values.astype(float)

log_arr2 = np.log(arr2)
mean_log2 = np.mean(log_arr2, axis=1, keepdims=True)

clr2 = log_arr2 - mean_log2

df2_clr = pd.DataFrame(
    clr2,
    columns=[c.replace('_ppm', '_clr') for c in REE2]
)
```

```
In [42]: proxy_cols2 = ['Y_ppm', 'TiO2_pct', 'MnO_pct']

X2_df = df2_clr.copy()

for col in proxy_cols2:
    X2_df[col] = np.log1p(df2[col])
```

```
In [43]: # FIXED: Scale AFTER CLR transformation
scaler2 = StandardScaler()
X2_scaled = scaler2.fit_transform(X2_df)

X2_scaled_df = pd.DataFrame(
    X2_scaled,
    columns=X2_df.columns
)

print("Dataset 2 preprocessing complete.")
print("Features:", X2_scaled_df.shape[1])
```

Dataset 2 preprocessing complete.
Features: 17

```
In [44]: autoencoder2 = MLPRegressor(
    hidden_layer_sizes=(10, 5, 10),
    activation='relu',
    solver='adam',
```

```
    max_iter=300,  
    random_state=42  
)  
  
autoencoder2.fit(X2_scaled_df, X2_scaled_df)
```

```

Out [44]: <style>.sk-global { /* Definition of color scheme common for light and dark mode */ --
sklearn-color-text: #000; --sklearn-color-text-muted: #666; --sklearn-color-line:
gray; /* Definition of color scheme for unfitted estimators */ --sklearn-color-unfitted-
level-0: #fff5e6; --sklearn-color-unfitted-level-1: #f6e4d2; --sklearn-color-unfitted-
level-2: #ffe0b3; --sklearn-color-unfitted-level-3: chocolate; /* Definition of color
scheme for fitted estimators */ --sklearn-color-fitted-level-0: #f0f8ff; --sklearn-color-
fitted-level-1: #d4ebff; --sklearn-color-fitted-level-2: #b3dbfd; --sklearn-color-
fitted-level-3: cornflowerblue; } .sk-global.light { /* Specific color for light theme */ --
sklearn-color-text-on-default-background: black; --sklearn-color-background: white;
--sklearn-color-border-box: black; --sklearn-color-icon: #696969; } .sk-global.dark {
--sklearn-color-text-on-default-background: white; --sklearn-color-background:
#111; --sklearn-color-border-box: white; --sklearn-color-icon: #878787; } .sk-global {
color: var(--sklearn-color-text); } .sk-global pre { padding: 0; } .sk-global input.sk-
hidden--visually { border: 0; clip-path: inset(100%); height: 1px; margin: -1px;
overflow: hidden; padding: 0; position: absolute; width: 1px; } .sk-global div.sk-
dashed-wrapped { border: 1px dashed var(--sklearn-color-line); margin: 0 0.4em
0.5em 0.4em; box-sizing: border-box; padding-bottom: 0.4em; background-color:
var(--sklearn-color-background); } .sk-global div.sk-container { /* jupyter's
`normalize.less` sets `[hidden] { display: none; }` but bootstrap.min.css set `[hidden]
{ display: none !important; }` so we also need the `!important` here to be able to
override the default hidden behavior on the sphinx rendered scikit-learn.org. See:
https://github.com/scikit-learn/scikit-learn/issues/21755 */ display: inline-block
!important; position: relative; } .sk-global div.sk-text-repr-fallback { display: none; }
div.sk-parallel-item, div.sk-serial, div.sk-item { /* draw centered vertical line to link
estimators */ background-image: linear-gradient(var(--sklearn-color-text-on-default-
background), var(--sklearn-color-text-on-default-background)); background-size:
2px 100%; background-repeat: no-repeat; background-position: center center; } /*
Parallel-specific style estimator block */ .sk-global div.sk-parallel-item::after { content:
""; width: 100%; border-bottom: 2px solid var(--sklearn-color-text-on-default-
background); flex-grow: 1; } .sk-global div.sk-parallel { display: flex; align-items:
stretch; justify-content: center; background-color: var(--sklearn-color-background);
position: relative; } .sk-global div.sk-parallel-item { display: flex; flex-direction:
column; } .sk-global div.sk-parallel-item:first-child::after { align-self: flex-end; width:
50%; } .sk-global div.sk-parallel-item:last-child::after { align-self: flex-start; width:
50%; } .sk-global div.sk-parallel-item:only-child::after { width: 0; } /* Serial-specific
style estimator block */ .sk-global div.sk-serial { display: flex; flex-direction: column;
align-items: center; background-color: var(--sklearn-color-background); padding-
right: 1em; padding-left: 1em; } /* Toggleable style: style used for
estimator/Pipeline/ColumnTransformer box that is clickable and can be
expanded/collapsed. - Pipeline and ColumnTransformer use this feature and define
the default style - Estimators will overwrite some part of the style using the `sk-
estimator` class */ /* Pipeline and ColumnTransformer style (default) */ .sk-global
div.sk-toggleable { /* Default theme specific background. It is overwritten whether we
have a specific estimator or a Pipeline/ColumnTransformer */ background-color: var(--
sklearn-color-background); } /* Toggleable label */ .sk-global label.sk-

```

```

toggleable__label { cursor: pointer; display: flex; width: 100%; margin-bottom: 0;
padding: 0.5em; box-sizing: border-box; text-align: center; align-items: center;
justify-content: center; gap: 0.5em; } .sk-global label.sk-toggleable__label .caption {
font-size: 0.6rem; font-weight: lighter; color: var(--sklearn-color-text-muted); } .sk-
global label.sk-toggleable__label-arrow:before { /* Arrow on the left of the label */
content: "▸"; float: left; margin-right: 0.25em; color: var(--sklearn-color-icon); } .sk-
global label.sk-toggleable__label-arrow:hover:before { color: var(--sklearn-color-
text); } /* Toggleable content - dropdown */ .sk-global div.sk-toggleable__content {
display: none; text-align: left; /* unfitted */ background-color: var(--sklearn-color-
unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted { /* fitted */
background-color: var(--sklearn-color-fitted-level-0); } .sk-global div.sk-
toggleable__content pre { margin: 0.2em; border-radius: 0.25em; color: var(--sklearn-
color-text); /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); }
.sk-global div.sk-toggleable__content.fitted pre { /* unfitted */ background-color: var(-
--sklearn-color-fitted-level-0); } .sk-global input.sk-
toggleable__control:checked~div.sk-toggleable__content { /* Expand drop-down */
display: block; width: 100%; overflow: visible; } .sk-global input.sk-
toggleable__control:checked~label.sk-toggleable__label-arrow:before { content: "▾"; }
/* Pipeline/ColumnTransformer-specific style */ .sk-global div.sk-label input.sk-
toggleable__control:checked~label.sk-toggleable__label { color: var(--sklearn-color-
text); background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-
label.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label {
background-color: var(--sklearn-color-fitted-level-2); } /* Estimator-specific style */ /*
Colorize estimator box */ .sk-global div.sk-estimator input.sk-
toggleable__control:checked~label.sk-toggleable__label { /* unfitted */ background-
color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted
input.sk-toggleable__control:checked~label.sk-toggleable__label { /* fitted */
background-color: var(--sklearn-color-fitted-level-2); } .sk-global div.sk-label
label.sk-toggleable__label, .sk-global div.sk-label label { /* The background is the
default theme color */ color: var(--sklearn-color-text-on-default-background); } /* On
hover, darken the color of the background */ .sk-global div.sk-label:hover label.sk-
toggleable__label { color: var(--sklearn-color-text); background-color: var(--sklearn-
color-unfitted-level-2); } /* Label box, darken color on hover, fitted */ .sk-global div.sk-
label.fitted:hover label.sk-toggleable__label.fitted { color: var(--sklearn-color-text);
background-color: var(--sklearn-color-fitted-level-2); } /* Estimator label */ .sk-global
div.sk-label label { font-family: monospace; font-weight: bold; line-height: 1.2em; }
.sk-global div.sk-label-container { text-align: center; } /* Estimator-specific */ .sk-
global div.sk-estimator { font-family: monospace; border: 1px dotted var(--sklearn-
color-border-box); border-radius: 0.25em; box-sizing: border-box; margin-bottom:
0.5em; /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-
global div.sk-estimator.fitted { /* fitted */ background-color: var(--sklearn-color-
fitted-level-0); } /* on hover */ .sk-global div.sk-estimator:hover { /* unfitted */
background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-
estimator.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-
2); } /* Specification for estimator info (e.g. "i" and "?") */ /* Common style for "i" and

```

```

"?" */ .sk-estimator-doc-link, a:link.sk-estimator-doc-link, a:visited.sk-estimator-doc-link { float: right; font-size: smaller; line-height: 1em; font-family: monospace; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 1em; height: 1em; width: 1em; text-decoration: none !important; margin-left: 0.5em; text-align: center; /* unfitted */ border: var(--sklearn-color-unfitted-level-3) 1pt solid; color: var(--sklearn-color-unfitted-level-3); } .sk-estimator-doc-link.fitted, a:link.sk-estimator-doc-link.fitted, a:visited.sk-estimator-doc-link.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-3) 1pt solid; color: var(--sklearn-color-fitted-level-3); } /* On hover */ div.sk-estimator:hover .sk-estimator-doc-link:hover, .sk-estimator-doc-link:hover, div.sk-label-container:hover .sk-estimator-doc-link:hover, .sk-estimator-doc-link:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid; color: var(--sklearn-color-unfitted-level-0); text-decoration: none; } div.sk-estimator.fitted:hover .sk-estimator-doc-link.fitted:hover, .sk-estimator-doc-link.fitted:hover, div.sk-label-container:hover .sk-estimator-doc-link.fitted:hover, .sk-estimator-doc-link.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid; color: var(--sklearn-color-fitted-level-0); text-decoration: none; } /* Span, style for the box shown on hovering the info icon */ .sk-estimator-doc-link span { display: none; z-index: 9999; position: relative; font-weight: normal; right: .2ex; padding: .5ex; margin: .5ex; width: min-content; min-width: 20ex; max-width: 50ex; color: var(--sklearn-color-text); box-shadow: 2pt 2pt 4pt #999; /* unfitted */ background: var(--sklearn-color-unfitted-level-0); border: .5pt solid var(--sklearn-color-unfitted-level-3); } .sk-estimator-doc-link.fitted span { /* fitted */ background: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-3); } .sk-estimator-doc-link:hover span { display: block; } /* "?"-specific style due to the `` HTML tag */ .sk-global a.estimator_doc_link { float: right; font-size: 1rem; line-height: 1em; font-family: monospace; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 1rem; height: 1rem; width: 1rem; text-decoration: none; /* unfitted */ color: var(--sklearn-color-unfitted-level-1); border: var(--sklearn-color-unfitted-level-1) 1pt solid; } .sk-global a.estimator_doc_link.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-1) 1pt solid; color: var(--sklearn-color-fitted-level-1); } /* On hover */ .sk-global a.estimator_doc_link:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-3); color: var(--sklearn-color-background); text-decoration: none; } .sk-global a.estimator_doc_link.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-3); } .sk-top-container.sk-global { /* pydata-sphinx-theme hides overflow, so scrolling is disabled. We need to set it to !important and add tabindex="0" in the HTML to allow keyboard-only users to navigate the display. */ overflow-x: scroll !important; max-width: 100%; } .estimator-table { font-family: monospace; } .estimator-table summary { padding: .5rem; cursor: pointer; } .estimator-table summary::marker { font-size: 0.7rem; } .estimator-table details[open] { padding-left: 0.1rem; padding-right: 0.1rem; padding-bottom: 0.3rem; } .estimator-table .parameters-table { margin-left: auto !important; margin-right: auto !important; margin-top: 0; } .estimator-table .parameters-table tr:nth-child(odd) {

```

```

background-color: #fff; } .estimator-table .parameters-table tr:nth-child(even) {
background-color: #ff6f6f; } .estimator-table .parameters-table tr:hover td {
background-color: #e0e0e0; } .estimator-table table :is(td, th) { border: 1px solid
rgba(106, 105, 104, 0.232); } /* `table td` is set in notebook with right text-align. We
need to overwrite it. */ .estimator-table table td.param { text-align: left; position:
relative; padding: 0; } .user-set td { color: rgb(255, 94, 0); text-align: left !important; }
.user-set td.value { color: rgb(255, 94, 0); background-color: transparent; } .default td,
.estimator-table th { color: black; text-align: left !important; } .user-set td i, .default td i
{ color: black; } td.fitted-att-type { white-space: preserve nowrap; } /* Styles for
parameter documentation links We need styling for visited so jupyter doesn't
overwrite it */ a.param-doc-link, a.param-doc-link:link, a.param-doc-link:visited {
text-decoration: underline dashed; text-underline-offset: .3em; color: inherit; display:
block; padding: .5em; } @supports(anchor-name: --doc-link) { a.param-doc-link,
a.param-doc-link:link, a.param-doc-link:visited { anchor-name: --doc-link; } } /*
"hack" to make the entire area of the cell containing the link clickable */ a.param-doc-
link::before { position: absolute; content: ""; inset: 0; } .param-doc-description {
display: none; position: absolute; z-index: 9999; left: 0; padding: .5ex; margin-left:
1.5em; color: var(--sklearn-color-text); box-shadow: .3em .3em .4em #999; width:
max-content; text-align: left; max-height: 10em; overflow-y: auto; /* unfitted */
background: var(--sklearn-color-unfitted-level-0); border: thin solid var(--sklearn-
color-unfitted-level-3); } @supports(position-area: center right) { .param-doc-
description { position-area: center right; position: fixed; margin-left: 0; } } /* Fitted
state for parameter tooltips */ .fitted .param-doc-description { /* fitted */ background:
var(--sklearn-color-fitted-level-0); border: thin solid var(--sklearn-color-fitted-level-
3); } .param-doc-link:hover .param-doc-description { display: block; } .copy-paste-
icon { background-image:
url(data:image/svg+xml;base64,PHN2ZyB4bWxucz0iaHR0cDovL3d3dy53My5vcmcvMjAv
background-repeat: no-repeat; background-size: 14px 14px; background-position: 0;
display: inline-block; width: 14px; height: 14px; cursor: pointer; } .features { font-
family: monospace; cursor: pointer; background-color: var(--sklearn-color-unfitted-
level-0); border: 1px dotted var(--sklearn-color-border-box); border-radius: .20em;
margin-bottom: 0.5em; font-size: inherit; /* Needed for jupyter */ } .features.fitted {
background-color: var(--sklearn-color-fitted-level-0); } .features.summary { cursor:
pointer; display: flex; margin-bottom: 0; text-align: center; align-items: center; justify-
content: center; gap: 0.5em; padding: .25em; } .features.details[open] > summary {
color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-
2); border-radius: .20em 0 0 0; } .features.fitted.details[open] > summary {
background-color: var(--sklearn-color-fitted-level-2); border-radius: .20em 0 0 0; }
.features.details > summary .arrow::before { content: "►"; color: grey; } .features
.details[open] > summary .arrow::before { content: "▼"; } .features.details:hover >
summary { margin: 0; background-color: var(--sklearn-color-unfitted-level-2); }
.features.fitted.details:hover > summary { margin: 0; background-color: var(--sklearn-
color-fitted-level-2); } .features .features-container { max-width: 15em; max-height:
10em; overflow: auto; scrollbar-width: thin; padding: .25em 0.1rem; background-color:
var(--sklearn-color-unfitted-level-0); border-radius: 0 0 .5em .5em; } .features.fitted

```

```
.features-container { background-color: var(--sklearn-color-fitted-level-0); } .features
.image-container { block-size: 1em; inline-size: 1em; padding: 0; margin: 0%; display:
flex; justify-content: center; align-items: center; } .features .copy-paste-icon {
background-size: 1em 1em; width: 1em; height: 1em; filter: grayscale(100%)
opacity(60%); } .features .features-container table { width: 100%; margin: 0.01em; }
.features .features-container table tr:nth-child(odd) { background-color: #fff; }
.features .features-container table tr:nth-child(even) { background-color: #f6f6f6; }
.features .features-container table tr:hover { background-color: #e0e0e0; } .features
.features-container table { table-layout: inherit; } .features .features-container table td
{ text-align: left; padding: 0 0.5em; border: 1px solid rgba(106, 105, 104, 0.232);
white-space: nowrap; color: var(--sklearn-color-text); } .total_features { display: flex;
justify-content: center; margin-top: 0.5em; } </style><body>
```

MLPRegressor(hidden_layer_sizes=(10, 5, 10), max_iter=300, random_state=42)

**In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.**

```
<input class="sk-toggleable__control sk-hidden--visually sk-global" id="sk-
estimator-id-3" type="checkbox" checked><label for="sk-estimator-id-3" class="sk-
toggleable__label fitted sk-toggleable__label-arrow">
```

MLPRegressor

[?Documentation for MLPRegressor](#)

```
</label>
```

```
<details> <summary>Parameters</summary> <tbody> </tbody> <tbody></tbody>
```

[hidden_layer_sizes](#) `hidden_layer_sizes`: array-like of shape(`n_layers - 2`), default=(100,)
The `ith` element represents the number of neurons in the `ith` hidden layer.

[max_iter](#) `max_iter`: int, default=200
Maximum number of iterations. The solver iterates until convergence (determined by 'tol') or this number of iterations. For stochastic solvers ('sgd', 'adam'), note that this determines the number of epochs (how many times each data point will be used), not the number of gradient steps.

[random_state](#) `random_state`: int, RandomState instance, default=None
Determines random number generation for weights and bias initialization, train-test split if early stopping is used, and batch sampling when solver='sgd' or 'adam'.
Pass an int for reproducible results across multiple function calls.
See [:term:`Glossary`](#) `<random_state>`.

[loss](#) `loss`: {'squared_error', 'poisson'}, default='squared_error'
The loss function to use when training the weights. Note that the "squared error" and "poisson" losses actually implement "half squares error" and "half poisson deviance" to simplify the computation of the gradient. Furthermore, the "poisson" loss internally uses a log-link (exponential as the output activation function) and requires `y >= 0`.
.. versionchanged:: 1.7
Added parameter 'loss' and option 'poisson'.

[activation](#) `activation`: {'identity', 'logistic', 'tanh', 'relu'}, default='relu'
Activation function for the hidden layer.
- 'identity', no-op activation, useful to implement linear bottleneck, returns $f(x) = x$
- 'logistic', the logistic sigmoid function, returns $f(x) = 1 / (1 + \exp(-x))$
- 'tanh', the hyperbolic tan function, returns $f(x) = \tanh(x)$
- 'relu', the rectified linear unit function, returns $f(x) = \max(0, x)$

`solver` solver: {'lbfgs', 'sgd', 'adam'}, default='adam'
The solver for weight optimization.
- 'lbfgs' is an optimizer in the family of quasi-Newton methods.
- 'sgd' refers to stochastic gradient descent.
- 'adam' refers to a stochastic gradient-based optimizer proposed by Kingma, Diederik, and Jimmy Ba
For a comparison between Adam optimizer and SGD, see: `sphinx_glr_auto_examples_neural_networks_plot_mlp_training_curves.py``.
Note: The default solver 'adam' works pretty well on relatively large datasets (with thousands of training samples or more) in terms of both training time and validation score. For small datasets, however, 'lbfgs' can converge faster and perform better.

`alpha` alpha: float, default=0.0001
Strength of the L2 regularization term. The L2 regularization term is divided by the sample size when added to the loss.

`batch_size` batch_size: int, default='auto'
Size of minibatches for stochastic optimizers. If the solver is 'lbfgs', the regressor will not use minibatch. When set to "auto", `batch_size=min(200, n_samples)``.

`learning_rate` learning_rate: {'constant', 'invscaling', 'adaptive'}, default='constant'
Learning rate schedule for weight updates.
- 'constant' is a constant learning rate given by `learning_rate_init``.
- 'invscaling' gradually decreases the learning rate `learning_rate`` at each time step 't' using an inverse scaling exponent of `power_t``.
$$\text{effective_learning_rate} = \text{learning_rate_init} / \text{pow}(t, \text{power_t})$$

- 'adaptive' keeps the learning rate constant to `learning_rate_init`` as long as training loss keeps decreasing. Each time two consecutive epochs fail to decrease training loss by at least `tol``, or fail to increase validation score by at least `tol`` if 'early_stopping' is on, the current learning rate is divided by 5. Only used when solver='sgd'.

`learning_rate_init` learning_rate_init: float, default=0.001
The initial learning rate used. It controls the step-size in updating the weights. Only used when solver='sgd' or 'adam'.

`power_t` power_t: float, default=0.5
The exponent for inverse scaling learning rate. It is used in updating effective learning rate when the learning_rate is set to 'invscaling'. Only used when solver='sgd'.

`shuffle` shuffle: bool, default=True
Whether to shuffle samples in each iteration. Only used when solver='sgd' or 'adam'.

`tol` tol: float, default=1e-4
Tolerance for the optimization. When the loss or score is not improving by at least `tol`` for `n_iter_no_change`` consecutive iterations, unless `learning_rate`` is set to 'adaptive', convergence is considered to be reached and training stops.

`verbose` verbose: bool, default=False
Whether to print progress messages to stdout.

`warm_start` warm_start: bool, default=False
When set to True, reuse the solution of the previous call to fit as initialization, otherwise, just erase the previous solution. See :term:`the Glossary <warm\_start>`.

`momentum` momentum: float, default=0.9
Momentum for gradient descent update. Should be between 0 and 1. Only used when solver='sgd'.

`nesterovs_momentum` nesterovs_momentum: bool, default=True
Whether to use Nesterov's momentum. Only used when solver='sgd' and momentum > 0.

`early_stopping` early_stopping: bool, default=False
Whether to use early stopping to terminate training when validation score is not improving. If set to True, it will automatically set aside `validation_fraction`` of training data as validation and terminate training when validation score is not improving by at least `tol`` for `n_iter_no_change`` consecutive epochs. Only effective when solver='sgd' or 'adam'.

validation_fraction validation_fraction: float, default=0.1
The proportion of training data to set aside as validation set for early stopping. Must be between 0 and 1. Only used if early_stopping is True.

beta_1 beta_1: float, default=0.9
Exponential decay rate for estimates of first moment vector in adam, should be in [0, 1). Only used when solver='adam'.

beta_2 beta_2: float, default=0.999
Exponential decay rate for estimates of second moment vector in adam, should be in [0, 1). Only used when solver='adam'.

epsilon epsilon: float, default=1e-8
Value for numerical stability in adam. Only used when solver='adam'.

n_iter_no_change n_iter_no_change: int, default=10
Maximum number of epochs to not meet ``tol`` improvement. Only effective when solver='sgd' or 'adam'.
.. versionadded:: 0.20

max_fun max_fun: int, default=15000
Only used when solver='lbfgs'. Maximum number of function calls. The solver iterates until convergence (determined by ``tol``), number of iterations reaches max_iter, or this number of function calls. Note that number of function calls will be greater than or equal to the number of iterations for the MLPRegressor.
.. versionadded:: 0.22

</details>

<details> <summary>Fitted attributes</summary> <tbody> <th>Name</th>
<th>Type</th> <th>Value</th> </tbody> <tbody></tbody>

best_loss_ best_loss_: float
The minimum loss reached by the solver throughout fitting.
If `early\_stopping=True`, this attribute is set to `None`. Refer to the `best\_validation\_score\_` fitted attribute instead.
Only accessible when solver='sgd' or 'adam'. float64

best_validation_score_ best_validation_score_: float or None
The best validation score (i.e. R2 score) that triggered the early stopping. Only available if `early\_stopping=True`, otherwise the attribute is set to `None`. Only accessible when solver='sgd' or 'adam'. NoneType

coefs_ coefs_: list of shape (n_layers - 1,)
The ith element in the list represents the weight matrix corresponding to layer i. list [array([[-4.35...9465588e-03]]), array([0.21348191]]), array([[0.38...-0.08157886]]), 9.11...7404275e

feature_names_in_ feature_names_in_: ndarray of shape (`n_features_in_` ,) Names of features seen during :term: `fit` . Defined only when `X` has feature names that are all strings. .. versionadded:: 1.0	ndarray[object] (17,)	['La_clr','Ce_clr','Pr_clr',..., 'Y_ppm','TiO2_pct','Mn
intercepts_ intercepts_: list of shape (n_layers - 1,) The ith element in the list represents the bias vector corresponding to layer i + 1.	list	[array([-0.448... 0.8325724]), array([-0.13567422]), array([-1.354... 0.0416 array([-0.279... 0.2603
loss_ loss_: float The current loss computed with the loss function.	float64	
loss_curve_ loss_curve_: list of shape (`n_iter_` ,) Loss value evaluated at the end of each training step. The ith element in the list represents the loss at the ith iteration. Only accessible when solver='sgd' or 'adam'.	list	[np.float64(0.580917718786 np.float64(0.553051606249 np.float64(0.53607799893 np.float64(0.52320483971469
n_features_in_ n_features_in_: int Number of features seen during :term: `fit` . .. versionadded:: 0.24	int	
n_iter_ n_iter_: int The number of iterations the solver has run.	int	
n_layers_ n_layers_: int Number of layers.	int	
n_outputs_ n_outputs_: int Number of outputs.	int	
out_activation_ out_activation_: str Name of the output activation function.	str	
t_ t_: int The number of training samples seen by the	int	s

solver during fitting.
 Mathematically
 equals ``n_iters * X.shape[0]``, it
 means ``time_step``
 and it is used by
 optimizer's learning rate
 scheduler.

`validation_scores_`
`validation_scores_`: list
 of shape `(`n_iter_`,)` or
 None
 The score
 at each iteration on a
 held-out validation set.
 The score reported
 is the R2 score. Only
 available if
``early_stopping=True``,
 otherwise the
 attribute is set to
``None``. Only
 accessible when
 solver='sgd' or 'adam'.

NoneType

</details>

```
<script>/* Authors: The scikit-learn developers SPDX-License-Identifier: BSD-3-
Clause */ function copyToClipboard(text, element) { // Get the parameter prefix from
the closest toggleable content const toggleableContent = element.closest('.sk-
toggleable__content'); const paramPrefix = toggleableContent ?
toggleableContent.dataset.paramPrefix : ''; const fullParamName = paramPrefix ? `
paramPrefix{text}` : text; const originalStyle = element.style; const computedStyle
= window.getComputedStyle(element); const originalWidth = computedStyle.width;
const originalHTML = element.innerHTML.replace('Copied!', '');
navigator.clipboard.writeText(fullParamName) .then(() => { element.style.width =
originalWidth; element.style.color = 'green'; element.innerHTML = "Copied!";
setTimeout(() => { element.innerHTML = originalHTML; element.style = originalStyle;
}, 2000); }) .catch(err => { console.error('Failed to copy:', err); element.style.color =
'red'; element.innerHTML = "Failed!"; setTimeout(() => { element.innerHTML =
originalHTML; element.style = originalStyle; }, 2000); }); return false; }
document.querySelectorAll('.copy-paste-icon').forEach(function(element) { const
toggleableContent = element.closest('.sk-toggleable__content'); const paramPrefix =
toggleableContent ? toggleableContent.dataset.paramPrefix : ''; const parent =
element.parentElement; if (!parent || !parent.nextElementSibling) {
console.warn('Expected copy-paste icon is missing from the DOM structure'); return;
} const paramName = element.parentElement.nextElementSibling
.textContent.trim().split(' ')[0]; const fullParamName = paramPrefix ? `paramPrefix
{paramName}` : paramName; element.setAttribute('title', fullParamName); }); /**
Copy the list of feature names formatted as a Python list. * * @param {HTMLElement}
element - The copy button inside a `.features` block; its siblings * contain a `details`
element and a table containing feature named. * @returns {boolean} Always returns
`false` so callers can prevent the default click behavior. */ function
```

```

copyFeatureNamesToClipboard(element) { var detailsElem =
element.closest('.features').querySelector('details'); var wasOpen = detailsElem.open;
detailsElem.open = true; var content =
element.closest('.features').querySelector('tbody') .innerText.trim(); if (!wasOpen)
detailsElem.open = false; const rows = content.split('\n').map(row => `
row ` `); const formattedText = `[\n{rows.join('\n')},\n]`; const originalHTML =
element.innerHTML.replace('✓', ''); const originalStyle = element.style; const
copyMark = document.createElement('span'); copyMark.innerHTML = '✓';
copyMark.style.color = 'blue'; copyMark.style.fontSize = '1em';
navigator.clipboard.writeText(formattedText) .then(() => { element.style.display =
'none'; element.parentElement.appendChild(copyMark); setTimeout(() => {
copyMark.remove(); element.innerHTML = originalHTML; element.style =
originalStyle; }, 1000); }) .catch(err => { console.error('Failed to copy:', err);
element.style.color = 'orange'; element.innerHTML = "Failed!"; setTimeout(() => {
element.innerHTML = originalHTML; element.style = originalStyle; }, 1000); }); return
false; } /** * Adapted from Skrub * https://github.com/skrub-
data/skrub/blob/403466d1d5d4dc76a7ef569b3f8228db59a31dc3/skrub/_reporting/_dat
* @returns "light" or "dark" */ function detectTheme(element) { const body =
document.querySelector('body'); // Check VSCode theme const themeKindAttr =
body.getAttribute('data-vscode-theme-kind'); const themeNameAttr =
body.getAttribute('data-vscode-theme-name'); if (themeKindAttr && themeNameAttr)
{ const themeKind = themeKindAttr.toLowerCase(); const themeName =
themeNameAttr.toLowerCase(); if (themeKind.includes("dark") ||
themeName.includes("dark")) { return "dark"; } if (themeKind.includes("light") ||
themeName.includes("light")) { return "light"; } } // Check Jupyter theme if
(body.getAttribute('data-jp-theme-light') === 'false') { return 'dark'; } else if
(body.getAttribute('data-jp-theme-light') === 'true') { return 'light'; } // Guess based
on a parent element's color const color =
window.getComputedStyle(element.parentNode, null).getPropertyValue('color');
const match = color.match(/^rgb\s*(\d+)\s*,\s*(\d+)\s*,\s*(\d+)\s*$/i); if (match) { const [r, g, b] = [
parseFloat(match[1]), parseFloat(match[2]),
parseFloat(match[3]) ]; // https://en.wikipedia.org/wiki/HSL_and_HSV#Lightness const
luma = 0.299 * r + 0.587 * g + 0.114 * b; if (luma > 180) { // If the text is very bright we
have a dark theme return 'dark'; } if (luma < 75) { // If the text is very dark we have a
light theme return 'light'; } // Otherwise fall back to the next heuristic. } // Fallback to
system preference return window.matchMedia('(prefers-color-scheme:
dark)').matches ? 'dark' : 'light'; } function forceTheme(elementId) { const
estimatorElement = document.querySelector(`#${elementId}`); if (estimatorElement
=== null) { console.error(`Element with id ${elementId} not found.`); } else { const
theme = detectTheme(estimatorElement); estimatorElement.classList.add(theme); } }
forceTheme('sk-container-id-3');
```

```

In [45]: X2_reconstructed = autoencoder2.predict(X2_scaled_df)

mse2 = np.mean(
    (X2_scaled_df.values - X2_reconstructed) ** 2,
    axis=1
)
```

```
)  
  
# Top 5% reconstruction errors = anomalies  
threshold2 = np.percentile(mse2, 95)  
  
y_true2 = (mse2 >= threshold2).astype(int)  
  
print("Anomalies:", y_true2.sum())  
print("Anomaly rate:", y_true2.mean())
```

Anomalies: 160
Anomaly rate: 0.05

```
In [46]: # FIXED: Stratified train/test split  
X_train2, X_test2, y_train2, y_test2 = train_test_split(  
    X2_scaled_df,  
    y_true2,  
    test_size=0.3,  
    random_state=42,  
    stratify=y_true2  
)
```

```
In [47]: # FIXED: Handle class imbalance  
scale_pos_weight2 = (  
    (y_train2 == 0).sum() /  
    (y_train2 == 1).sum()  
)  
  
xgb_clf2 = xgb.XGBClassifier(  
    n_estimators=100,  
    max_depth=3,  
    random_state=42,  
    verbosity=0,  
    scale_pos_weight=scale_pos_weight2  
)  
  
xgb_clf2.fit(X_train2, y_train2)
```

```

Out [47]: <style>.sk-global { /* Definition of color scheme common for light and dark mode */ --
sklearn-color-text: #000; --sklearn-color-text-muted: #666; --sklearn-color-line:
gray; /* Definition of color scheme for unfitted estimators */ --sklearn-color-unfitted-
level-0: #fff5e6; --sklearn-color-unfitted-level-1: #f6e4d2; --sklearn-color-unfitted-
level-2: #ffe0b3; --sklearn-color-unfitted-level-3: chocolate; /* Definition of color
scheme for fitted estimators */ --sklearn-color-fitted-level-0: #f0f8ff; --sklearn-color-
fitted-level-1: #d4ebff; --sklearn-color-fitted-level-2: #b3dbfd; --sklearn-color-
fitted-level-3: cornflowerblue; } .sk-global.light { /* Specific color for light theme */ --
sklearn-color-text-on-default-background: black; --sklearn-color-background: white;
--sklearn-color-border-box: black; --sklearn-color-icon: #696969; } .sk-global.dark {
--sklearn-color-text-on-default-background: white; --sklearn-color-background:
#111; --sklearn-color-border-box: white; --sklearn-color-icon: #878787; } .sk-global {
color: var(--sklearn-color-text); } .sk-global pre { padding: 0; } .sk-global input.sk-
hidden--visually { border: 0; clip-path: inset(100%); height: 1px; margin: -1px;
overflow: hidden; padding: 0; position: absolute; width: 1px; } .sk-global div.sk-
dashed-wrapped { border: 1px dashed var(--sklearn-color-line); margin: 0 0.4em
0.5em 0.4em; box-sizing: border-box; padding-bottom: 0.4em; background-color:
var(--sklearn-color-background); } .sk-global div.sk-container { /* jupyter's
`normalize.less` sets `[hidden] { display: none; }` but bootstrap.min.css set `[hidden]
{ display: none !important; }` so we also need the `!important` here to be able to
override the default hidden behavior on the sphinx rendered scikit-learn.org. See:
https://github.com/scikit-learn/scikit-learn/issues/21755 */ display: inline-block
!important; position: relative; } .sk-global div.sk-text-repr-fallback { display: none; }
div.sk-parallel-item, div.sk-serial, div.sk-item { /* draw centered vertical line to link
estimators */ background-image: linear-gradient(var(--sklearn-color-text-on-default-
background), var(--sklearn-color-text-on-default-background)); background-size:
2px 100%; background-repeat: no-repeat; background-position: center center; } /*
Parallel-specific style estimator block */ .sk-global div.sk-parallel-item::after { content:
""; width: 100%; border-bottom: 2px solid var(--sklearn-color-text-on-default-
background); flex-grow: 1; } .sk-global div.sk-parallel { display: flex; align-items:
stretch; justify-content: center; background-color: var(--sklearn-color-background);
position: relative; } .sk-global div.sk-parallel-item { display: flex; flex-direction:
column; } .sk-global div.sk-parallel-item:first-child::after { align-self: flex-end; width:
50%; } .sk-global div.sk-parallel-item:last-child::after { align-self: flex-start; width:
50%; } .sk-global div.sk-parallel-item:only-child::after { width: 0; } /* Serial-specific
style estimator block */ .sk-global div.sk-serial { display: flex; flex-direction: column;
align-items: center; background-color: var(--sklearn-color-background); padding-
right: 1em; padding-left: 1em; } /* Toggleable style: style used for
estimator/Pipeline/ColumnTransformer box that is clickable and can be
expanded/collapsed. - Pipeline and ColumnTransformer use this feature and define
the default style - Estimators will overwrite some part of the style using the `sk-
estimator` class */ /* Pipeline and ColumnTransformer style (default) */ .sk-global
div.sk-toggleable { /* Default theme specific background. It is overwritten whether we
have a specific estimator or a Pipeline/ColumnTransformer */ background-color: var(--
sklearn-color-background); } /* Toggleable label */ .sk-global label.sk-

```

```

toggleable__label { cursor: pointer; display: flex; width: 100%; margin-bottom: 0;
padding: 0.5em; box-sizing: border-box; text-align: center; align-items: center;
justify-content: center; gap: 0.5em; } .sk-global label.sk-toggleable__label .caption {
font-size: 0.6rem; font-weight: lighter; color: var(--sklearn-color-text-muted); } .sk-
global label.sk-toggleable__label-arrow:before { /* Arrow on the left of the label */
content: "▸"; float: left; margin-right: 0.25em; color: var(--sklearn-color-icon); } .sk-
global label.sk-toggleable__label-arrow:hover:before { color: var(--sklearn-color-
text); } /* Toggleable content - dropdown */ .sk-global div.sk-toggleable__content {
display: none; text-align: left; /* unfitted */ background-color: var(--sklearn-color-
unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted { /* fitted */
background-color: var(--sklearn-color-fitted-level-0); } .sk-global div.sk-
toggleable__content pre { margin: 0.2em; border-radius: 0.25em; color: var(--sklearn-
color-text); /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); }
.sk-global div.sk-toggleable__content.fitted pre { /* unfitted */ background-color: var(-
--sklearn-color-fitted-level-0); } .sk-global input.sk-
toggleable__control:checked~div.sk-toggleable__content { /* Expand drop-down */
display: block; width: 100%; overflow: visible; } .sk-global input.sk-
toggleable__control:checked~label.sk-toggleable__label-arrow:before { content: "▾"; }
/* Pipeline/ColumnTransformer-specific style */ .sk-global div.sk-label input.sk-
toggleable__control:checked~label.sk-toggleable__label { color: var(--sklearn-color-
text); background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-
label.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label {
background-color: var(--sklearn-color-fitted-level-2); } /* Estimator-specific style */ /*
Colorize estimator box */ .sk-global div.sk-estimator input.sk-
toggleable__control:checked~label.sk-toggleable__label { /* unfitted */ background-
color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted
input.sk-toggleable__control:checked~label.sk-toggleable__label { /* fitted */
background-color: var(--sklearn-color-fitted-level-2); } .sk-global div.sk-label
label.sk-toggleable__label, .sk-global div.sk-label label { /* The background is the
default theme color */ color: var(--sklearn-color-text-on-default-background); } /* On
hover, darken the color of the background */ .sk-global div.sk-label:hover label.sk-
toggleable__label { color: var(--sklearn-color-text); background-color: var(--sklearn-
color-unfitted-level-2); } /* Label box, darken color on hover, fitted */ .sk-global div.sk-
label.fitted:hover label.sk-toggleable__label.fitted { color: var(--sklearn-color-text);
background-color: var(--sklearn-color-fitted-level-2); } /* Estimator label */ .sk-global
div.sk-label label { font-family: monospace; font-weight: bold; line-height: 1.2em; }
.sk-global div.sk-label-container { text-align: center; } /* Estimator-specific */ .sk-
global div.sk-estimator { font-family: monospace; border: 1px dotted var(--sklearn-
color-border-box); border-radius: 0.25em; box-sizing: border-box; margin-bottom:
0.5em; /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-
global div.sk-estimator.fitted { /* fitted */ background-color: var(--sklearn-color-
fitted-level-0); } /* on hover */ .sk-global div.sk-estimator:hover { /* unfitted */
background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-
estimator.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-
2); } /* Specification for estimator info (e.g. "i" and "?") */ /* Common style for "i" and

```

```

"?" */ .sk-estimator-doc-link, a:link.sk-estimator-doc-link, a:visited.sk-estimator-doc-link { float: right; font-size: smaller; line-height: 1em; font-family: monospace; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 1em; height: 1em; width: 1em; text-decoration: none !important; margin-left: 0.5em; text-align: center; /* unfitted */ border: var(--sklearn-color-unfitted-level-3) 1pt solid; color: var(--sklearn-color-unfitted-level-3); } .sk-estimator-doc-link.fitted, a:link.sk-estimator-doc-link.fitted, a:visited.sk-estimator-doc-link.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-3) 1pt solid; color: var(--sklearn-color-fitted-level-3); } /* On hover */ div.sk-estimator:hover .sk-estimator-doc-link:hover, .sk-estimator-doc-link:hover, div.sk-label-container:hover .sk-estimator-doc-link:hover, .sk-estimator-doc-link:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid; color: var(--sklearn-color-unfitted-level-0); text-decoration: none; } div.sk-estimator.fitted:hover .sk-estimator-doc-link.fitted:hover, .sk-estimator-doc-link.fitted:hover, div.sk-label-container:hover .sk-estimator-doc-link.fitted:hover, .sk-estimator-doc-link.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid; color: var(--sklearn-color-fitted-level-0); text-decoration: none; } /* Span, style for the box shown on hovering the info icon */ .sk-estimator-doc-link span { display: none; z-index: 9999; position: relative; font-weight: normal; right: .2ex; padding: .5ex; margin: .5ex; width: min-content; min-width: 20ex; max-width: 50ex; color: var(--sklearn-color-text); box-shadow: 2pt 2pt 4pt #999; /* unfitted */ background: var(--sklearn-color-unfitted-level-0); border: .5pt solid var(--sklearn-color-unfitted-level-3); } .sk-estimator-doc-link.fitted span { /* fitted */ background: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-3); } .sk-estimator-doc-link:hover span { display: block; } /* "?"-specific style due to the `` HTML tag */ .sk-global a.estimator_doc_link { float: right; font-size: 1rem; line-height: 1em; font-family: monospace; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 1rem; height: 1rem; width: 1rem; text-decoration: none; /* unfitted */ color: var(--sklearn-color-unfitted-level-1); border: var(--sklearn-color-unfitted-level-1) 1pt solid; } .sk-global a.estimator_doc_link.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-1) 1pt solid; color: var(--sklearn-color-fitted-level-1); } /* On hover */ .sk-global a.estimator_doc_link:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-3); color: var(--sklearn-color-background); text-decoration: none; } .sk-global a.estimator_doc_link.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-3); } .sk-top-container.sk-global { /* pydata-sphinx-theme hides overflow, so scrolling is disabled. We need to set it to !important and add tabindex="0" in the HTML to allow keyboard-only users to navigate the display. */ overflow-x: scroll !important; max-width: 100%; } .estimator-table { font-family: monospace; } .estimator-table summary { padding: .5rem; cursor: pointer; } .estimator-table summary::marker { font-size: 0.7rem; } .estimator-table details[open] { padding-left: 0.1rem; padding-right: 0.1rem; padding-bottom: 0.3rem; } .estimator-table .parameters-table { margin-left: auto !important; margin-right: auto !important; margin-top: 0; } .estimator-table .parameters-table tr:nth-child(odd) {

```

```

background-color: #fff; } .estimator-table .parameters-table tr:nth-child(even) {
background-color: #ff6f6f; } .estimator-table .parameters-table tr:hover td {
background-color: #e0e0e0; } .estimator-table table :is(td, th) { border: 1px solid
rgba(106, 105, 104, 0.232); } /* `table td` is set in notebook with right text-align. We
need to overwrite it. */ .estimator-table table td.param { text-align: left; position:
relative; padding: 0; } .user-set td { color: rgb(255, 94, 0); text-align: left !important; }
.user-set td.value { color: rgb(255, 94, 0); background-color: transparent; } .default td,
.estimator-table th { color: black; text-align: left !important; } .user-set td i, .default td i
{ color: black; } td.fitted-att-type { white-space: preserve nowrap; } /* Styles for
parameter documentation links We need styling for visited so jupyter doesn't
overwrite it */ a.param-doc-link, a.param-doc-link:link, a.param-doc-link:visited {
text-decoration: underline dashed; text-underline-offset: .3em; color: inherit; display:
block; padding: .5em; } @supports(anchor-name: --doc-link) { a.param-doc-link,
a.param-doc-link:link, a.param-doc-link:visited { anchor-name: --doc-link; } } /*
"hack" to make the entire area of the cell containing the link clickable */ a.param-doc-
link::before { position: absolute; content: ""; inset: 0; } .param-doc-description {
display: none; position: absolute; z-index: 9999; left: 0; padding: .5ex; margin-left:
1.5em; color: var(--sklearn-color-text); box-shadow: .3em .3em .4em #999; width:
max-content; text-align: left; max-height: 10em; overflow-y: auto; /* unfitted */
background: var(--sklearn-color-unfitted-level-0); border: thin solid var(--sklearn-
color-unfitted-level-3); } @supports(position-area: center right) { .param-doc-
description { position-area: center right; position: fixed; margin-left: 0; } } /* Fitted
state for parameter tooltips */ .fitted .param-doc-description { /* fitted */ background:
var(--sklearn-color-fitted-level-0); border: thin solid var(--sklearn-color-fitted-level-
3); } .param-doc-link:hover .param-doc-description { display: block; } .copy-paste-
icon { background-image:
url(data:image/svg+xml;base64,PHN2ZyB4bWxucz0iaHR0cDovL3d3dy53My5vcmcvMjAv
background-repeat: no-repeat; background-size: 14px 14px; background-position: 0;
display: inline-block; width: 14px; height: 14px; cursor: pointer; } .features { font-
family: monospace; cursor: pointer; background-color: var(--sklearn-color-unfitted-
level-0); border: 1px dotted var(--sklearn-color-border-box); border-radius: .20em;
margin-bottom: 0.5em; font-size: inherit; /* Needed for jupyter */ } .features.fitted {
background-color: var(--sklearn-color-fitted-level-0); } .features.summary { cursor:
pointer; display: flex; margin-bottom: 0; text-align: center; align-items: center; justify-
content: center; gap: 0.5em; padding: .25em; } .features.details[open] > summary {
color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-
2); border-radius: .20em 0 0 0; } .features.fitted.details[open] > summary {
background-color: var(--sklearn-color-fitted-level-2); border-radius: .20em 0 0 0; }
.features.details > summary .arrow::before { content: "►"; color: grey; } .features
.details[open] > summary .arrow::before { content: "▼"; } .features.details:hover >
summary { margin: 0; background-color: var(--sklearn-color-unfitted-level-2); }
.features.fitted.details:hover > summary { margin: 0; background-color: var(--sklearn-
color-fitted-level-2); } .features .features-container { max-width: 15em; max-height:
10em; overflow: auto; scrollbar-width: thin; padding: .25em 0.1rem; background-color:
var(--sklearn-color-unfitted-level-0); border-radius: 0 0 .5em .5em; } .features.fitted

```

```
.features-container { background-color: var(--sklearn-color-fitted-level-0); } .features
.image-container { block-size: 1em; inline-size: 1em; padding: 0; margin: 0%; display:
flex; justify-content: center; align-items: center; } .features .copy-paste-icon {
background-size: 1em 1em; width: 1em; height: 1em; filter: grayscale(100%)
opacity(60%); } .features .features-container table { width: 100%; margin: 0.01em; }
.features .features-container table tr:nth-child(odd) { background-color: #fff; }
.features .features-container table tr:nth-child(even) { background-color: #f6f6f6; }
.features .features-container table tr:hover { background-color: #e0e0e0; } .features
.features-container table { table-layout: inherit; } .features .features-container table td
{ text-align: left; padding: 0 0.5em; border: 1px solid rgba(106, 105, 104, 0.232);
white-space: nowrap; color: var(--sklearn-color-text); } .total_features { display: flex;
justify-content: center; margin-top: 0.5em; } </style><body>
```

```
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=None,
              enable_categorical=True, eval_metric=None, feature_types=None,
              feature_weights=None, gamma=None, grow_policy=None,
              importance_type=None, interaction_constraints=None,
              learning_rate=None, max_bin=None, max_cat_threshold=None,
              max_cat_to_onehot=None, max_delta_step=None, max_depth=3,
              max_leaves=None, min_child_weight=None, missing=nan,
              monotone_constraints=None, multi_strategy=None, n_estimators=100,
              n_jobs=None, num_parallel_tree=None, ...)
```

**In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.**

```
<input class="sk-toggleable__control sk-hidden--visually sk-global" id="sk-
estimator-id-4" type="checkbox" checked><label for="sk-estimator-id-4" class="sk-
toggleable__label fitted sk-toggleable__label-arrow">
```

XGBClassifier

[?Documentation for XGBClassifier](#)

```
</label>
```

```
<details> <summary>Parameters</summary> <tbody> </tbody> <tbody></tbody>
```

base_score	base_score: typing.Union[float, typing.List[float], NoneType]	 The initial prediction score of all instances, global bias.	None
------------	---	---	------

booster	None
---------	------

callbacks	callbacks: typing.Optional[typing.List[xgboost.callback.TrainingCallback]]	 List of callback functions that are applied at end of each iteration. It is possible to use predefined callbacks by using :ref: `Callback API <callback_api>`. .. note:: States in callback are not preserved during training, which means callback objects can not be reused for multiple training sessions without reinitialization or deepcopy. .. code-block:: python for params in parameters_grid: # be sure to (re)initialize the callbacks before each run callbacks = [xgb.callback.LearningRateScheduler(custom_rates)] reg =	None
-----------	--	---	------

<pre>xgboost.XGBRegressor(**params, callbacks=callbacks)
 reg.fit(X, y)</pre>	
<pre>colsample_bylevel colsample_bylevel: typing.Optional[float]

Subsample ratio of columns for each level.</pre>	None
<pre>colsample_bynode colsample_bynode: typing.Optional[float]

Subsample ratio of columns for each split.</pre>	None
<pre>colsample_bytree colsample_bytree: typing.Optional[float]

Subsample ratio of columns when constructing each tree.</pre>	None
<pre>device device: typing.Optional[str]

.. versionadded:: 2.0.0

Device ordinal, available options are `cpu`, `cuda`, and `gpu`.</pre>	None
<pre>early_stopping_rounds early_stopping_rounds: typing.Optional[int]

.. versionadded:: 1.6.0

- Activates early stopping. Validation metric needs to improve at least once in
 every **early_stopping_rounds** round(s) to continue training. Requires at
 least one item in **eval_set** in :py:meth: `fit`.

- If early stopping occurs, the model will have two additional attributes:
 :py:attr: `best_score` and :py:attr: `best_iteration`. These are used by the
 :py:meth: `predict` and :py:meth: `apply` methods to determine the optimal
 number of trees during inference. If users want to access the full model
 (including trees built after early stopping), they can specify the
 `iteration_range` in these inference methods. In addition, other utilities
 like model plotting can also use the entire model.

- If you prefer to discard the trees after `best_iteration`, consider using the
 callback function :py:class: `xgboost.callback.EarlyStopping`.

- If there's more than one item in **eval_set**, the last entry will be used for
 early stopping. If there's more than one metric in **eval_metric**, the last
 metric will be used for early stopping.</pre>	None
<pre>enable_categorical enable_categorical: bool

See the same parameter of :py:class: `DMatrix` for details.</pre>	True
<pre>eval_metric eval_metric: typing.Union[str, typing.List[typing.Union[str, typing.Callable]], typing.Callable, NoneType]

.. versionadded:: 1.6.0

Metric used for monitoring the training result and early stopping. It can be a
string or list of strings as names of predefined metric in XGBoost (See
:doc: `/parameter`), one of the metrics in :py:mod: `sklearn.metrics`, or any
other user defined metric that looks like `sklearn.metrics`.

If custom objective is also provided, then custom metric should implement the
corresponding reverse link function.

Unlike the `scoring` parameter commonly used in scikit-learn, when a callable
object is provided, it's assumed to be a cost function and by default XGBoost
will minimize the result during early stopping.

For advanced usage on Early stopping like directly choosing to maximize instead
of minimize, see :py:obj: `xgboost.callback.EarlyStopping`.

See :doc: `/tutorials/custom_metric_obj` and :ref: `custom-obj-metric` for more
information.

.. code-block:: python

 from sklearn.datasets import load_diabetes
 from sklearn.metrics import mean_absolute_error
 X, y = load_diabetes(return_X_y=True)
 reg = xgb.XGBRegressor(
 tree_method="hist",
 eval_metric=mean_absolute_error,
)
 reg.fit(X, y, eval_set= [(X, y)])</pre>	None
<pre>feature_types feature_types: typing.Optional[typing.Sequence[str]]

.. versionadded:: 1.7.0

Used for specifying feature</pre>	None

types without constructing a dataframe. See [the :py:class:`DMatrix`](#) for details.

feature_weights feature_weights: Optional[ArrayLike] Weight for each feature, defines the probability of each feature being selected when colsample is being used. All values must be greater than 0, otherwise a <code>ValueError</code> is thrown.	None
gamma gamma: typing.Optional[float] (min_split_loss) Minimum loss reduction required to make a further partition on a leaf node of the tree.	None
grow_policy grow_policy: typing.Optional[str] Tree growing policy. - depthwise: Favors splitting at nodes closest to the node, - lossguide: Favors splitting at nodes with highest loss change.	None
importance_type	None
interaction_constraints interaction_constraints: typing.Union[str, typing.List[typing.Tuple[str]], NoneType] Constraints for interaction representing permitted interactions. The constraints must be specified in the form of a nested list, e.g. <code>[[0, 1], [2, 3, 4]]</code> , where each inner list is a group of indices of features that are allowed to interact with each other. See :doc:`tutorial/tutorials/feature_interaction_constraint` for more information	None
learning_rate learning_rate: typing.Optional[float] Boosting learning rate (xgb's "eta")	None
max_bin max_bin: typing.Optional[int] If using histogram-based algorithm, maximum number of bins per feature	None
max_cat_threshold max_cat_threshold: typing.Optional[int] .. versionadded:: 1.7.0 .. note:: This parameter is experimental Maximum number of categories considered for each split. Used only by partition-based splits for preventing over-fitting. Also, <code>enable_categorical</code> needs to be set to have categorical feature support. See :doc:`Categorical Data/tutorials/categorical` and :ref:`cat-param` for details.	None
max_cat_to_onehot max_cat_to_onehot: Optional[int] .. versionadded:: 1.6.0 .. note:: This parameter is experimental A threshold for deciding whether XGBoost should use one-hot encoding based split for categorical data. When number of categories is lesser than the threshold then one-hot encoding is chosen, otherwise the categories will be partitioned into children nodes. Also, <code>enable_categorical</code> needs to be set to have categorical feature support. See :doc:`Categorical Data/tutorials/categorical` and :ref:`cat-param` for details.	None
max_delta_step max_delta_step: typing.Optional[float] Maximum delta step we allow each tree's weight estimation to be.	None
max_depth max_depth: typing.Optional[int] Maximum tree depth for base learners.	3
max_leaves max_leaves: typing.Optional[int] Maximum number of leaves; 0 indicates no limit.	None
min_child_weight min_child_weight: typing.Optional[float] Minimum sum of instance weight(hessian) needed in a child.	None

missing missing: float Value in the data which needs to be present as a missing value. Default to :py:data:`numpy.nan`.	nan
monotone_constraints monotone_constraints: typing.Union[typing.Dict[str, int], str, NoneType] Constraint of variable monotonicity. See :doc:`tutorial </tutorials/monotonic>` for more information.	None
multi_strategy multi_strategy: typing.Optional[str] .. versionadded:: 2.0.0 .. note:: This parameter is working-in-progress. The strategy used for training multi-target models, including multi-target regression and multi-class classification. See :doc:`/tutorials/multioutput` for more information. - ``one_output_per_tree``: One model for each target. - ``multi_output_tree``: Use multi-target trees.	None
n_estimators n_estimators: Optional[int] Number of boosting rounds.	100
n_jobs n_jobs: typing.Optional[int] Number of parallel threads used to run xgboost. When used with other Scikit-Learn algorithms like grid search, you may choose which algorithm to parallelize and balance the threads. Creating thread contention will significantly slow down both algorithms.	None
num_parallel_tree	None
random_state random_state: typing.Union[numpy.random.mtrand.RandomState, numpy.random._generator.Generator, int, NoneType] Random number seed. .. note:: Using gblinear booster with shotgun updater is nondeterministic as it uses Hogwild algorithm.	42
reg_alpha reg_alpha: typing.Optional[float] L1 regularization term on weights (xgb's alpha).	None
reg_lambda reg_lambda: typing.Optional[float] L2 regularization term on weights (xgb's lambda).	None
sampling_method sampling_method: typing.Optional[str] Sampling method. Used only by the GPU version of ``hist`` tree method. - ``uniform``: Select random training instances uniformly. - ``gradient_based``: Select random training instances with higher probability when the gradient and hessian are larger. (cf. CatBoost)	None
scale_pos_weight scale_pos_weight: typing.Optional[float] Balancing of positive and negative weights.	np.float64(19.0)
subsample subsample: typing.Optional[float] Subsample ratio of the training instance.	None
tree_method tree_method: typing.Optional[str] Specify which tree method to use. Default to auto. If this parameter is set to default, XGBoost will choose the most conservative option available. It's recommended to study this option from the parameters document :doc:`tree method </treemethod>`	None
validate_parameters validate_parameters: typing.Optional[bool] Give warnings for unknown parameter.	None
verbosity verbosity: typing.Optional[int] The degree of verbosity. Valid values are 0 (silent) - 3 (debug).	0

</details>

classes_	ndarray[int64] (2,)
feature_importances_	ndarray[float32] (17,) [0.06,0.03,0.06,...,0.05,0.03,0.03]
feature_names_in_	ndarray[<U8] (17,) ['La_clr','Ce_clr','Pr_clr',...,'Y_ppm','TiO2_pct','MnO2_pct']
intercept_	ndarray[float32] (1,)
n_classes_	int
n_features_in_	int

</details>

33/36

```

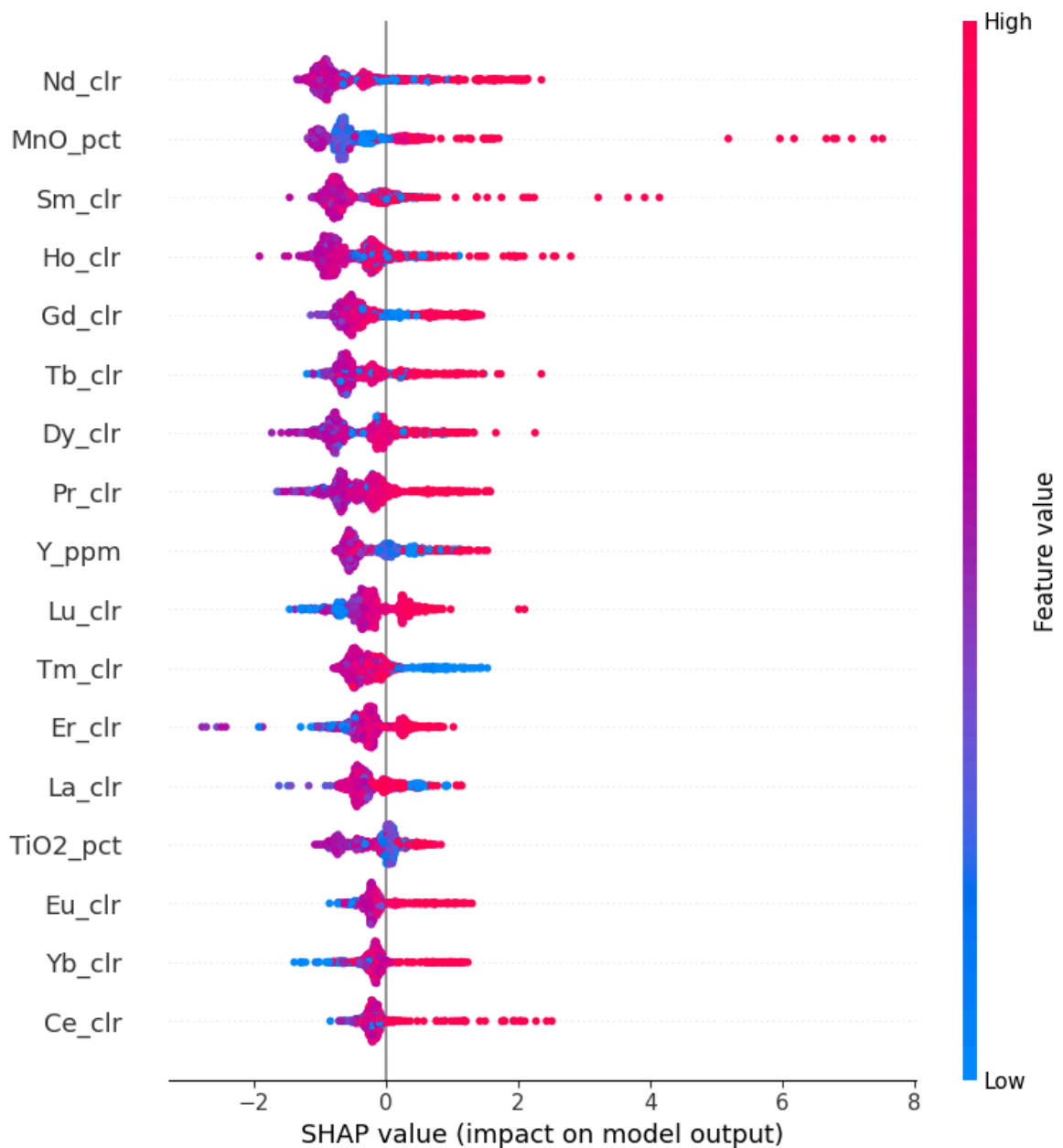
element - The copy button inside a `.features` block; its siblings * contain a `details`
element and a table containing feature named. * @returns {boolean} Always returns
`false` so callers can prevent the default click behavior. */ function
copyFeatureNamesToClipboard(element) { var detailsElem =
element.closest('.features').querySelector('details'); var wasOpen = detailsElem.open;
detailsElem.open = true; var content =
element.closest('.features').querySelector('tbody') .innerText.trim(); if (!wasOpen)
detailsElem.open = false; const rows = content.split('\n').map(row => `
row` ` `); const formattedText = `[\n{rows.join('\n')},\n]`; const originalHTML =
element.innerHTML.replace('✓', ''); const originalStyle = element.style; const
copyMark = document.createElement('span'); copyMark.innerHTML = '✓';
copyMark.style.color = 'blue'; copyMark.style.fontSize = '1em';
navigator.clipboard.writeText(formattedText) .then(() => { element.style.display =
'none'; element.parentElement.appendChild(copyMark); setTimeout(() => {
copyMark.remove(); element.innerHTML = originalHTML; element.style =
originalStyle; }, 1000); }) .catch(err => { console.error('Failed to copy:', err);
element.style.color = 'orange'; element.innerHTML = "Failed!"; setTimeout(() => {
element.innerHTML = originalHTML; element.style = originalStyle; }, 1000); }); return
false; } /** * Adapted from Skrub * https://github.com/skrub-
data/skrub/blob/403466d1d5d4dc76a7ef569b3f8228db59a31dc3/skrub/_reporting/_dat
* @returns "light" or "dark" */ function detectTheme(element) { const body =
document.querySelector('body'); // Check VSCode theme const themeKindAttr =
body.getAttribute('data-vscode-theme-kind'); const themeNameAttr =
body.getAttribute('data-vscode-theme-name'); if (themeKindAttr && themeNameAttr)
{ const themeKind = themeKindAttr.toLowerCase(); const themeName =
themeNameAttr.toLowerCase(); if (themeKind.includes("dark") ||
themeName.includes("dark")) { return "dark"; } if (themeKind.includes("light") ||
themeName.includes("light")) { return "light"; } } // Check Jupyter theme if
(body.getAttribute('data-jp-theme-light') === 'false') { return 'dark'; } else if
(body.getAttribute('data-jp-theme-light') === 'true') { return 'light'; } // Guess based
on a parent element's color const color =
window.getComputedStyle(element.parentNode, null).getPropertyValue('color');
const match = color.match(/^rgb\s*(\d+)\s*,\s*(\d+)\s*,\s*(\d+)\s*$/i); if (match) { const [r, g, b] = [
parseFloat(match[1]), parseFloat(match[2]),
parseFloat(match[3]) ]; // https://en.wikipedia.org/wiki/HSL_and_HSV#Lightness const
luma = 0.299 * r + 0.587 * g + 0.114 * b; if (luma > 180) { // If the text is very bright we
have a dark theme return 'dark'; } if (luma < 75) { // If the text is very dark we have a
light theme return 'light'; } // Otherwise fall back to the next heuristic. } // Fallback to
system preference return window.matchMedia('(prefers-color-scheme:
dark)').matches ? 'dark' : 'light'; } function forceTheme(elementId) { const
estimatorElement = document.querySelector(`#${elementId}`); if (estimatorElement
=== null) { console.error(`Element with id ${elementId} not found.`); } else { const
theme = detectTheme(estimatorElement); estimatorElement.classList.add(theme); } }
forceTheme('sk-container-id-4');
```

```
In [48]: # Predictions
y_pred2 = xgb_clf2.predict(X_test2)
y_prob2 = xgb_clf2.predict_proba(X_test2)[:, 1]

# SHAP
explainer2 = shap.TreeExplainer(xgb_clf2)
shap_values2 = explainer2.shap_values(X_test2)

shap.summary_plot(
    shap_values2,
    X_test2,
    show=False
)

plt.tight_layout()
plt.show()
```



```
In [49]: precision2 = precision_score(y_test2, y_pred2)
recall2 = recall_score(y_test2, y_pred2)
f1_2 = f1_score(y_test2, y_pred2)
roc_auc2 = roc_auc_score(y_test2, y_prob2)
```

```

avg_precision2 = average_precision_score(y_test2, y_prob2)

print("\n===== DATASET 2 RESULTS =====")
print(f"Precision:           {precision2:.3f}")
print(f"Recall:              {recall2:.3f}")
print(f"F1 Score:             {f1_2:.3f}")
print(f"ROC AUC:              {roc_auc2:.3f}")
print(f"Average Precision:   {avg_precision2:.3f}")

```

```

===== DATASET 2 RESULTS =====
Precision:           0.633
Recall:             0.396
F1 Score:           0.487
ROC AUC:            0.917
Average Precision:   0.550

```

<p>--</p> <h2 id="Submission-Checklist">Submission Checklist</h2><p>Before submitting, verify:</p>

- [*] All 5 errors are fixed in Section 2 and Section 4
- [*] Each fix has a `# FIXED:` comment explaining the change
- [*] All 6 reflection questions (Q1–Q6) are answered in your own words
- [*] The validation cell (Section 6) shows 5 marks
- [*] Dataset 2 pipeline (Section 8) runs and produces metrics
- [*] You can verbally explain any fix if asked by the instructor

In []:

In []:

In []: